

INSTITUTO FEDERAL DO TRIÂNGULO MINEIRO

Campus Uberlândia Centro

Estruturas de Dados

Tecnologia em Sistemas para Internet — 2026/1

AULA PRÁTICA

Tabelas Hash

Visão geral da aula prática

Esta aula tem duração de 3 horas e está organizada em três blocos de 60 minutos. O primeiro bloco trata de funções hash, contrato hashCode()/equals() e encadeamento separado. O segundo bloco cobre endereçamento aberto e rehashing. O terceiro bloco aplica os conceitos em problemas contextualizados de sistemas web.

Bloco	Tópico principal	Tipo	Duração
1	hashCode()/equals(); encadeamento separado	Fixação	60 min
2	Endereçamento aberto; rehashing; TabelaHash<K,V>	Fixação	60 min
3	Problemas contextualizados (sistemas web)	Aplicação	60 min

Pré-requisito: o aluno deve ter lido a apostila sobre tabela hash e ter o projeto Maven configurado com JUnit 5 e JMH.

Entrega ao final da aula: repositório Git com os exercícios resolvidos e a suíte JUnit 5 passando. Não é necessário terminar os desafios de casa durante a aula.

Nota sobre a produção deste material

Assistência por IA: Esta apostila foi produzida com o auxílio do Claude (Anthropic), modelo de inteligência artificial generativa, sob supervisão direta e revisão sistemática do professor responsável pela disciplina.

Processo de criação: O professor definiu a estrutura pedagógica, os objetivos de aprendizagem e o escopo de cada seção. O Claude gerou os rascunhos iniciais do conteúdo técnico e os exemplos de código, com base nas referências bibliográficas indicadas na Seção 12 (Cormen et al., Sedgewick, Knuth e documentação oficial do OpenJDK). Todo o material gerado foi submetido a revisão crítica, validação técnica dos exemplos em Java e ajustes pedagógicos pelo professor. A formatação final foi gerada via script Python (python-docx).

Integridade acadêmica: O uso de inteligência artificial na produção deste material é declarado em conformidade com os princípios de transparência e integridade acadêmica. A responsabilidade pelo conteúdo final é do professor responsável pela disciplina.

BLOCO 1 — hashCode(), equals() e encadeamento separado (60 min)

Exercício 1.1 — Contrato hashCode()/equals() (15 min)

O código abaixo contém três classes com contratos hashCode()/equals() incorretos ou incompletos. Para cada classe, identifique o problema, corrija o código e explique por que a versão original quebra o funcionamento de HashMap e HashSet.

Classe A — localizar o erro

```
// Classe A
public class Produto {
    private String sku;
    private String nome;

    public Produto(String sku, String nome) {
        this.sku = sku;
        this.nome = nome;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Produto)) return false;
        Produto p = (Produto) o;
        return sku.equals(p.sku);
    }
    // hashCode() não foi sobrescrito
}
```

Classe B — localizar o erro

```
// Classe B
public class Sessao {
    private final String token;

    public Sessao(String token) { this.token = token; }

    @Override
    public int hashCode() { return 42; } // sempre retorna a mesma constante

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Sessao)) return false;
        return token.equals(((Sessao) o).token);
    }
}
```

Classe C — localizar o erro

```
// Classe C
public class Coordenada {
    public double lat;
    public double lon;

    public Coordenada(double lat, double lon) {
        this.lat = lat;
        this.lon = lon;
    }
}
```

```

    }

    @Override
    public int hashCode() {
        return Objects.hash(lat, lon);
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Coordenada)) return false;
        Coordenada c = (Coordenada) o;
        return lat == c.lat && lon == c.lon;    // comparação de double com ==
    }
}

```

Exercício 1.2 — Rastreamento manual: encadeamento separado (20 min)

Considere uma tabela hash com capacidade $M = 7$ usando encadeamento separado. A função hash é $h(k) = k \% 7$. Execute manualmente as operações abaixo na ordem indicada, desenhando o estado da tabela após cada inserção.

Sequência de inserções: inserir(10), inserir(22), inserir(31), inserir(4), inserir(15), inserir(28), inserir(17)

Chave	$h(k) = k \% 7$	Bucket	Colisão?
10	$10 \% 7 = 3$	3	Não
22	$22 \% 7 = 1$	1	Não
31	$31 \% 7 = 3$	3	Sim — encadeia com 10
4	$4 \% 7 = 4$	4	Não
15	$15 \% 7 = 1$	1	Sim — encadeia com 22
28	$28 \% 7 = 0$	0	Não
17	$17 \% 7 = 3$	3	Sim — encadeia com 10→31

Responda às perguntas a seguir:

1. Qual é o fator de carga $\lambda = n/M$ após todas as inserções?
2. Qual é o comprimento da cadeia mais longa? Qual operação tem custo $O(n)$ no pior caso quando isso acontece?
3. Se inseríssemos a chave 38, em qual bucket ela seria alocada? Haveria colisão?
4. A função $h(k) = k \% 7$ distribui bem as chaves desta sequência? Proponha uma sequência de 7 chaves que cause a pior distribuição possível para essa função.

Exercício 1.3 — Implementação: TabelaHashEncadeada<K,V> (25 min)

Implemente os métodos put(), get() e remove() da classe abaixo. Os comentários indicam o comportamento esperado. Escreva também os três testes JUnit 5 descritos no final do trecho.

Esqueleto a completar

```
public class TabelaHashEncadeada<K, V> {

    private static final int CAPACIDADE_PADRAO = 16;

    private LinkedList<Entrada<K, V>>[] buckets;
    private int tamanho;

    @SuppressWarnings("unchecked")
    public TabelaHashEncadeada(int capacidade) {
        buckets = new LinkedList[capacidade];
        for (int i = 0; i < capacidade; i++) {
            buckets[i] = new LinkedList<>();
        }
        this.tamanho = 0;
    }

    public TabelaHashEncadeada() {
        this(CAPACIDADE_PADRAO);
    }

    // Retorna o índice do bucket para a chave k.
    // Trate k == null explicitamente (lance IllegalArgumentException).
    private int indiceBucket(K k) {
        // TODO
    }

    // Insere ou atualiza o par (chave, valor).
    // Se a chave já existir, substitui o valor e NÃO incrementa tamanho.
    public void put(K chave, V valor) {
        // TODO
    }

    // Retorna o valor associado à chave, ou null se não encontrado.
    public V get(K chave) {
        // TODO
    }

    // Remove o par com a chave dada. Retorna true se removido, false se não encontrado.
    public boolean remove(K chave) {
        // TODO
    }

    public int tamanho() { return tamanho; }
    public boolean estaVazia() { return tamanho == 0; }

    // Classe interna de entrada
    private static class Entrada<K, V> {
        K chave;
        V valor;
        Entrada(K chave, V valor) { this.chave = chave; this.valor = valor; }
    }
}
```

Testes JUnit 5 a implementar

```
class TabelaHashEncadeadaTest {

    private TabelaHashEncadeada<String, Integer> tabela;
```

```

@BeforeEach
void setUp() { tabela = new TabelaHashEncadeada<>(4); }

@Test
@DisplayName("put e get: chave inserida deve ser recuperada")
void testPutGet() {
    // TODO: inserir "produto-42" → 100; recuperar e verificar
}

@Test
@DisplayName("put com atualização: tamanho não deve aumentar")
void testAtualizacaoNaoIncrementaTamanho() {
    // TODO: inserir mesma chave duas vezes com valores diferentes;
    //         verificar que tamanho() == 1 e que get retorna o valor mais recente
}

@Test
@DisplayName("colisão: duas chaves no mesmo bucket devem ser recuperadas corretamente")
void testColisao() {
    // Dica: com capacidade 4, qualquer duas chaves cujo hashCode() % 4 seja igual
    // serão alocadas no mesmo bucket.
    // TODO
}
}

```

Referência de implementação: `indiceBucket` deve usar `Math.abs(chave.hashCode()) % buckets.length` para evitar índice negativo, pois `hashCode()` pode retornar valores negativos em Java.

BLOCO 2 — Endereçamento aberto, rehashing e TabelaHash<K,V> genérica (60 min)

Exercício 2.1 — Rastreamento manual: sondagem linear e quadrática (20 min)

Considere uma tabela hash com endereçamento aberto, capacidade $M = 11$, e função hash base $h(k) = k \% 11$. Trace a inserção das chaves abaixo para cada estratégia de sondagem.

Sequência: inserir(20), inserir(31), inserir(54), inserir(43), inserir(65), inserir(9)

Parte A — Sondagem linear: $h(k, i) = (h(k) + i) \% M$

Chave	$h(k)$	Sonda $i=0$	Sonda $i=1$	Sonda $i=2$	Slot final
20	$20 \% 11 = 9$	9 livre	—	—	9
31	$31 \% 11 = 9$	9 ocupado	10 livre	—	10
54	$54 \% 11 = 10$	10 ocupado	0 livre	—	0
43	$43 \% 11 = 10$	10 ocupado	0 ocupado	1 livre	1
65	$65 \% 11 = 10$	10 ocupado	0 ocupado	1 ocup.	...
9	$9 \% 11 = 9$	9 ocupado	10 ocup.	0 ocup.	...

Complete as duas últimas linhas e responda:

- Qual é o fenômeno que ocorre quando chaves com o mesmo $h(k)$ inicial formam um bloco contíguo? Como a sondagem quadrática atenua esse problema?
- Calcule o fator de carga após todas as inserções. O valor está dentro do limite recomendado para endereçamento aberto?

Exercício 2.2 — Double hashing (15 min)

Utilize double hashing com as funções abaixo para inserir as chaves 76, 40, 48, 5, 55, 47 em uma tabela de capacidade $M = 11$.

Funções de sondagem

```
h1(k) = k % 11
h2(k) = 7 - (k % 7)           // passo secundário; 7 é primo < M
h(k, i) = (h1(k) + i * h2(k)) % 11
```

Para cada chave, calcule h_1 , h_2 e o slot final. Mostre as sondas intermediárias quando houver colisão.

Exercício 2.3 — Rehashing: análise e implementação (25 min)

O trecho abaixo implementa `put()` em uma tabela com endereçamento aberto por sondagem linear, mas não possui rehashing. Sua tarefa é:

- Identificar quando o rehashing deve ser disparado (defina o limiar de fator de carga).

8. Implementar o método `rehash()` que dobra a capacidade e reinserere todos os pares existentes.
9. Escrever dois testes JUnit 5: um que verifica que o rehashing é disparado quando $\lambda \geq 0,70$, e outro que confirma que todos os pares são corretamente migrados para a nova tabela.

Trecho base (endereçamento aberto, sondagem linear)

```
public class TabelaHashAberta<K, V> {

    // Marcador de slot deletado (lazy deletion)
    private static final Object DELETADO = new Object();

    private Object[] chaves;
    private Object[] valores;
    private int tamanho;
    private int capacidade;

    public TabelaHashAberta(int capacidade) {
        this.capacidade = capacidade;
        this.chaves = new Object[capacidade];
        this.valores = new Object[capacidade];
        this.tamanho = 0;
    }

    private int sonda(K chave, int i) {
        return (Math.abs(chave.hashCode()) + i) % capacidade;
    }

    public void put(K chave, V valor) {
        // Não dispara rehashing – IMPLEMENTE ESSA VERIFICAÇÃO
        for (int i = 0; i < capacidade; i++) {
            int idx = sonda(chave, i);
            if (chaves[idx] == null || chaves[idx] == DELETADO) {
                chaves[idx] = chave;
                valores[idx] = valor;
                tamanho++;
                return;
            }
            if (chaves[idx].equals(chave)) {
                valores[idx] = valor;    // atualização
                return;
            }
        }
        throw new IllegalStateException("Tabela cheia");
    }

    // IMPLEMENTE: dobra capacidade e reinserere todos os pares válidos
    private void rehash() {
        // TODO
    }

    public double fatorDeCarga() { return (double) tamanho / capacidade; }
}
```

BLOCO 3 — Problemas contextualizados em sistemas web (60 min)

Os três problemas a seguir simulam situações reais de desenvolvimento com Spring Boot. Para cada um, leia o enunciado com atenção, escreva o código Java e responda às perguntas de análise.

Problema 3.1 — Cache de sessões HTTP (20 min)

Um sistema de autenticação precisa armazenar tokens de sessão em memória com acesso $O(1)$. O token é uma String UUID, e o valor é um objeto Usuario com campos id (long) e perfil (String). A tabela deve:

- Recuperar um usuário por token em tempo $O(1)$ amortizado.
- Invalidar (remover) a sessão de um token específico.
- Verificar se um token ainda é válido (está presente na tabela).

Implemente a classe GerenciadorSessao utilizando sua TabelaHashEncadeada<String, Usuario> do Exercício 1.3.

Esqueleto

```
public class GerenciadorSessao {

    private final TabelaHashEncadeada<String, Usuario> sessoes;

    public GerenciadorSessao() {
        // Defina uma capacidade inicial adequada e justifique.
        // TODO
    }

    // Registra nova sessão. Lança IllegalArgumentException se token for nulo ou vazio.
    public void registrar(String token, Usuario usuario) { /* TODO */ }

    // Retorna o Usuario ou null se a sessão não existir.
    public Usuario buscar(String token) { /* TODO */ }

    // Remove a sessão. Retorna true se existia, false se já estava expirada/inválida.
    public boolean invalidar(String token) { /* TODO */ }

    // Retorna true se o token ainda é válido.
    public boolean eValido(String token) { /* TODO */ }
}
```

Perguntas de análise:

10. Qual é a complexidade esperada (caso médio) de registrar(), buscar() e invalidar() com encadeamento separado e fator de carga $\lambda \leq 0,75$?
11. Por que String é uma boa escolha como chave de tabela hash em Java? Quais propriedades da classe String garantem isso?
12. Em produção, esse cache em memória apresentaria qual problema crítico em ambientes com múltiplos servidores? Qual tecnologia resolveria isso?

Problema 3.2 — Índice invertido para busca de produtos (20 min)

Um sistema de e-commerce precisa de busca por palavra-chave em nomes de produtos. A estrutura escolhida é um índice invertido: cada palavra mapeia para a lista de IDs dos produtos que a contêm.

Exemplo: produto 1 = 'cadeira ergonômica', produto 2 = 'mesa de escritório', produto 3 = 'cadeira gamer'. O índice deve conter: 'cadeira' → [1, 3], 'ergonômica' → [1], 'mesa' → [2], 'escritório' → [2], 'gamer' → [3].

Esqueleto a completar

```
public class IndiceInvertido {

    // chave: palavra normalizada (minúscula, sem acentos)
    // valor: lista de IDs de produtos que contêm a palavra
    private final Map<String, List<Long>> indice = new HashMap<>();

    // Indexa o produto. Normaliza o nome antes de dividir em palavras.
    public void indexar(long idProduto, String nomeProduto) {
        // Dica: use nomeProduto.toLowerCase().split("\\s+") para dividir em palavras.
        // TODO
    }

    // Retorna a lista de IDs que contêm a palavra, ou lista vazia se não encontrada.
    public List<Long> buscar(String palavra) {
        // TODO
    }

    // Retorna os IDs que aparecem em TODAS as palavras da consulta (interseção).
    // Exemplo: buscar("cadeira gamer") → [3]
    public List<Long> buscarMultiplas(String consulta) {
        // TODO
    }

    public int totalPalavrasIndexadas() { return indice.size(); }
}
```

Perguntas de análise:

13. Qual é a complexidade de `indexar()` em função do número de palavras W do nome do produto?
14. Qual é a complexidade de `buscarMultiplas()` em função do número de palavras na consulta Q e do tamanho máximo de uma lista de resultados R ?
15. `HashMap<String, List<Long>>` vs `TreeMap<String, List<Long>>`: qual escolher para este índice e por quê? Em que caso `TreeMap` seria preferível?

Problema 3.3 — Rate limiting por IP com janela deslizante (20 min)

Uma API REST precisa limitar requisições por IP: no máximo 100 requisições por minuto por cliente. A estratégia é janela deslizante simplificada: para cada IP, armazene um contador e o timestamp do início da janela atual.

Esqueleto a completar

```
public class RateLimiter {
```

```

private static final int LIMITE = 100;
private static final long JANELA_MS = 60_000L;    // 1 minuto em milissegundos

// chave: IP do cliente (String)
// valor: registro com contador e início da janela
private final Map<String, Janela> contadores = new HashMap<>();

// Retorna true se a requisição for permitida, false se o limite foi atingido.
public boolean permitir(String ip) {
    long agora = System.currentTimeMillis();
    Janela janela = contadores.get(ip);

    if (janela == null || agora - janela.inicioMs >= JANELA_MS) {
        // Janela expirou ou IP novo – reinicia a janela
        contadores.put(ip, new Janela(agora, 1));
        return true;
    }

    if (janela.contador < LIMITE) {
        janela.contador++;
        return true;
    }

    return false; // limite atingido
}

// IMPLEMENTE: remove entradas de IPs cuja janela já expirou.
// Útil para evitar crescimento ilimitado da tabela em produção.
public void limparExpirados() {
    // TODO: iterar sobre entrySet() e remover entradas com janela expirada.
    // Dica: não modifique o Map durante iteração direta; use removeIf ou
    iterator.remove()
}

private static class Janela {
    long inicioMs;
    int contador;
    Janela(long inicioMs, int contador) {
        this.inicioMs = inicioMs;
        this.contador = contador;
    }
}
}

```

Perguntas de análise:

16. Qual é a complexidade de permitir() no caso médio? E de limparExpirados() em função do número N de IPs ativos?
17. Por que não se deve usar Map.entrySet().forEach() com map.remove() dentro do lambda para limpar entradas? Qual exceção seria lançada?
18. Em um sistema com 1 milhão de IPs distintos por hora, a tabela pode crescer indefinidamente. Além de limparExpirados(), cite uma estrutura de dados do JCF que gera expiração automática por tempo de acesso e que costuma ser base para implementações de LRU cache.

Desafios para casa

Os desafios abaixo são opcionais durante a aula, mas obrigatórios como entrega na semana seguinte. Todos exigem suíte JUnit 5.

Desafio 1 — HashMap: internals e treeification (nível médio)

O HashMap do Java 8+ converte um bucket de lista encadeada em uma árvore vermelho-preta quando a cadeia supera 8 elementos (TREEIFY_THRESHOLD). Esse mecanismo é chamado treeification.

19. Escreva um programa que provoque deliberadamente a treeification em um `HashMap<Integer, String>` inserindo chaves com o mesmo `hashCode()`. Dica: crie uma classe `ChaveColisora` com `hashCode()` fixo.
20. Use reflexão (`getDeclaredField`) para inspecionar o array interno do HashMap e verificar se o bucket foi convertido em `TreeNode`. Imprima o tipo do primeiro elemento do bucket antes e depois da treeification.
21. Meça com `System.nanoTime()` o tempo médio de `get()` antes e após a treeification para confirmar a melhora de $O(n)$ para $O(\log n)$. Use pelo menos 1000 repetições para obter medição estável.

Referência: `OpenJDK HashMap.java`, linha ~234 (constante `TREEIFY_THRESHOLD`) e método `treeifyBin()`. Leitura direta do código-fonte é a melhor forma de entender o funcionamento interno.

Desafio 2 — LRU Cache com LinkedHashMap (nível médio)

Implemente um cache LRU (Least Recently Used) de capacidade K utilizando `LinkedHashMap` com `accessOrder=true`. O cache deve:

- Armazenar pares (String chave, Object valor).
- Ter complexidade $O(1)$ para `get()` e `put()`.
- Remover automaticamente o par menos recentemente acessado quando a capacidade for atingida.
- Expor um método `List<String> ordemDeAcesso()` que retorna as chaves do mais antigo para o mais recente.

Escreva testes JUnit 5 verificando: inserção até o limite, evicção do item menos recente ao inserir além do limite, promoção de um item para o topo da ordem ao ser acessado com `get()`.

Desafio 3 — Deduplicação de logs em streaming (nível avançado)

Um sistema de ingestão de logs recebe eventos com campo `requestId` (String UUID). Por falhas de rede, o mesmo evento pode chegar duplicado em um intervalo de até 5 minutos. O sistema deve descartar duplicatas nesse intervalo.

Implemente `DeduplicadorLogs` com as seguintes restrições:

1. Use apenas estruturas do JCF (HashMap, LinkedHashMap, LinkedList) — não use bibliotecas externas.
2. O método boolean processar(String requestId, long timestampMs) deve retornar true se o evento for novo (primeira ocorrência), false se for duplicata dentro de 5 minutos.
3. Entradas com timestamp de mais de 5 minutos atrás devem ser removidas automaticamente durante a chamada a processar() para evitar crescimento ilimitado da memória.
4. A complexidade de processar() deve ser $O(1)$ amortizado.

Dica de projeto: mantenha um `HashMap<String, Long>` com `requestId` → `timestamp` e um `LinkedList<String>` com a ordem de inserção. Durante cada `processar()`, remova do início da fila entradas cujo `timestamp` seja mais velho que 5 minutos.

Desafio 4 — Benchmark: HashMap vs TabelaHashEncadeada (nível avançado)

Use JMH para comparar o throughput de `put()` e `get()` entre seu `TabelaHashEncadeada<String, Integer>` e o `HashMap<String, Integer>` do JCF nas seguintes condições:

- 100.000 pares, chaves geradas aleatoriamente.
- Capacidade inicial igual à quantidade de pares (sem rehashing durante o benchmark).
- Medir separadamente: inserção em ordem aleatória, busca de chaves existentes, busca de chaves inexistentes.

Documente os resultados em uma tabela e explique os fatores que contribuem para a diferença de desempenho (cache de CPU, autoboxing, treeification, qualidade da JVM JIT).

Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introdução a algoritmos. 4. ed. Rio de Janeiro: Elsevier, 2024. Cap. 11 (Tabelas Hash).

GOODRICH, M. T.; TAMASSIA, R. Estruturas de dados e algoritmos em Java. 6. ed. Porto Alegre: Bookman, 2013. Cap. 10.

BLOCH, J. Java efetivo. 3. ed. Rio de Janeiro: Alta Books, 2019. Item 11: Always override hashCode when you override equals.

ORACLE. HashMap (Java SE 21). Disponível em: <https://docs.oracle.com/en/java/docs/api/java.base/java/util/HashMap.html>. Acesso em: maio 2026.

ORACLE. LinkedHashMap (Java SE 21). Disponível em: <https://docs.oracle.com/en/java/docs/api/java.base/java/util/LinkedHashMap.html>. Acesso em: maio 2026.

OPENJDK. HashMap.java source. Disponível em: <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/util/HashMap.java>. Acesso em: maio 2026.

Glossário rápido

Termo	Definição
Bucket	Posição de armazenamento na tabela; no encadeamento separado, contém uma lista de pares.
Fator de carga (λ)	Razão n/M entre o número de pares armazenados e a capacidade da tabela.
Colisão	Situação em que duas chaves distintas produzem o mesmo índice de bucket.
Encadeamento separado	Estratégia de colisão que mantém listas (ou árvores) em cada bucket.
Endereçamento aberto	Estratégia de colisão que busca o próximo slot livre dentro do próprio array.
Sondagem linear	Endereçamento aberto em que cada tentativa avança um slot: $h(k,i) = (h(k)+i) \% M$.
Clustering primário	Agrupamento de chaves em blocos contíguos causado pela sondagem linear.
Double hashing	Endereçamento aberto que usa um segundo hash como passo: $h(k,i) = (h_1(k)+i*h_2(k)) \% M$.
Rehashing	Redimensionamento da tabela (normalmente dobrando M) com reinserção de todos os pares.
Treeification	Conversão de bucket-lista em bucket-árvore no HashMap do Java 8+ quando cadeia > 8 .
Lazy deletion	Marcação de slot como deletado (tombstone) sem remoção imediata, usada no endereçamento aberto.
LRU cache	Cache que descarta o item menos recentemente acessado quando atinge a capacidade máxima.